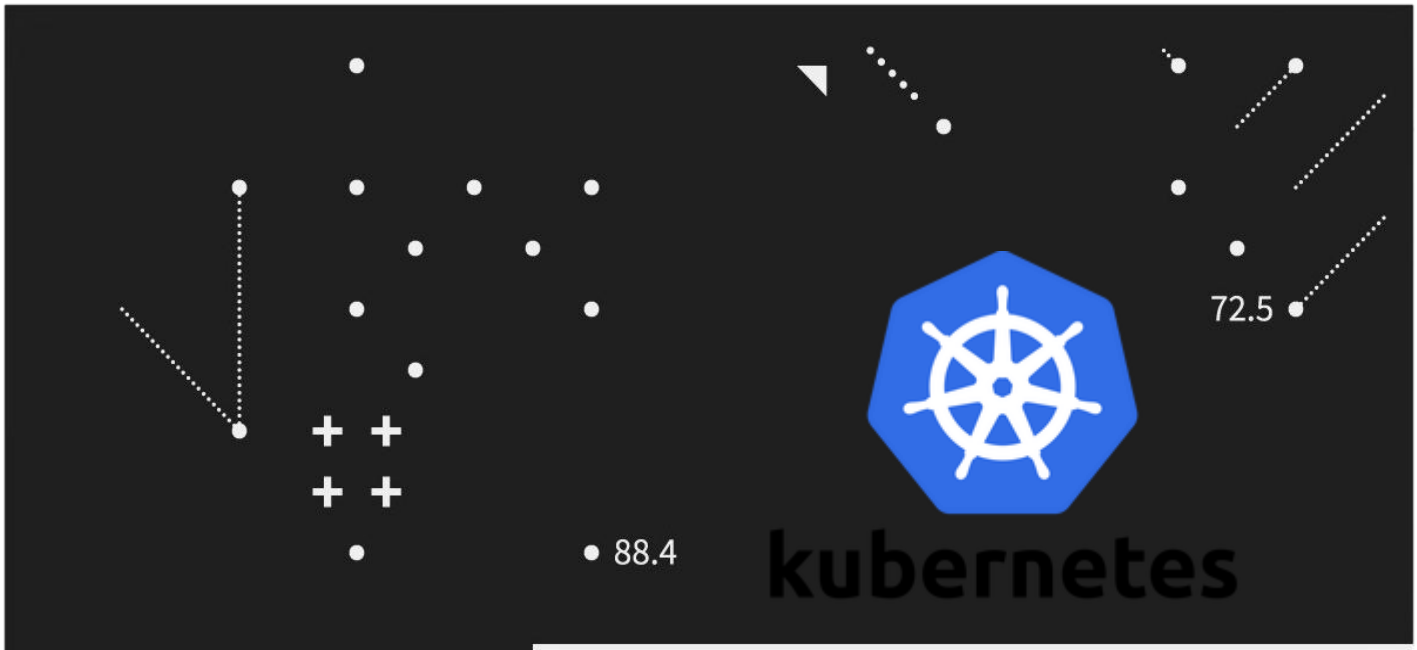
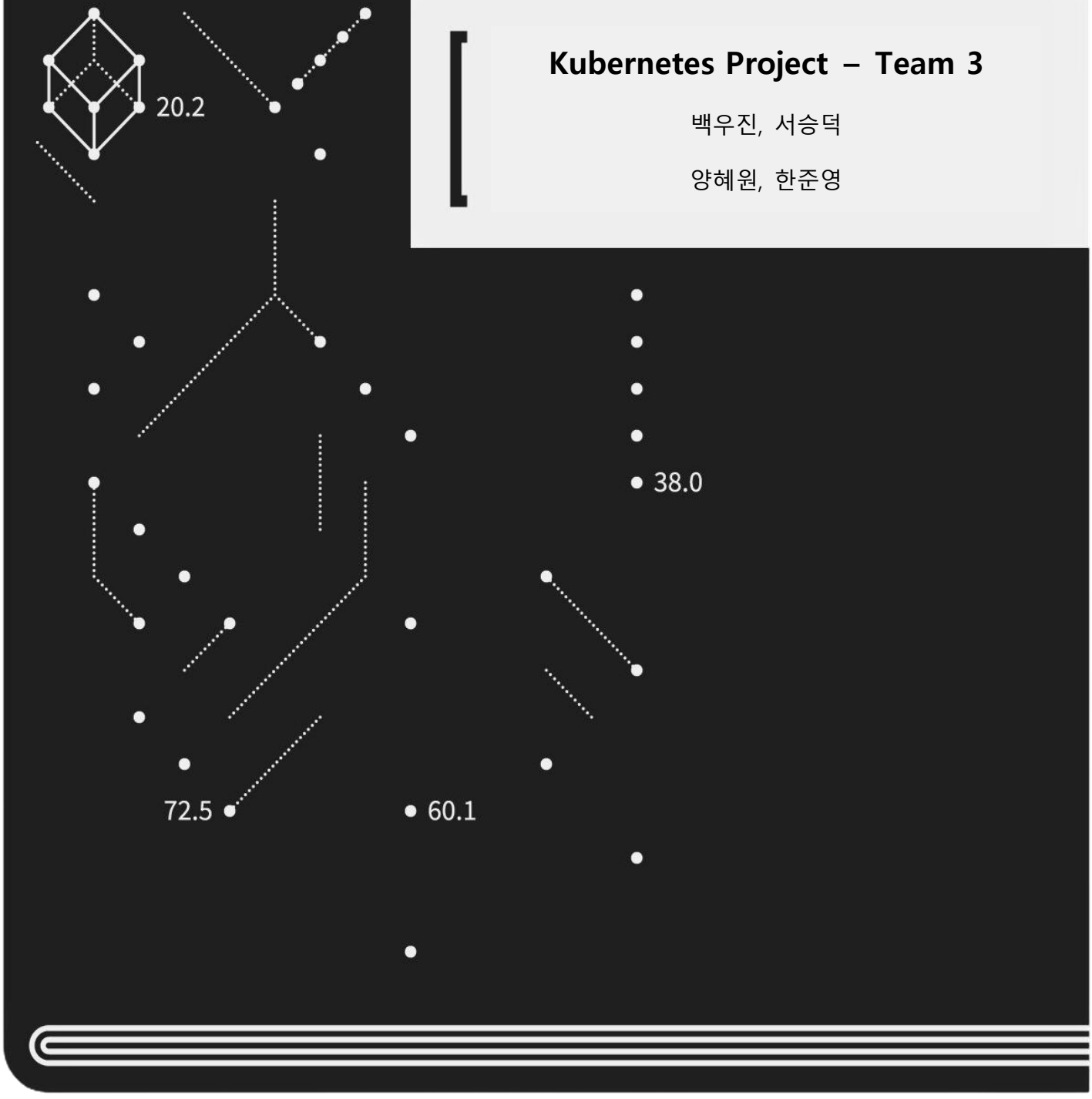


Kubernetes Project – Team 3

백우진, 서승덕

양혜원, 한준영



[문제 1] ETCD 백업 및 업그레이드

[문제 1-1] ETCD 백업

https://127.0.0.1:2379에서 실행 중인 etcd의 snapshot을 생성하고 snapshot을 etcd-snapshot.db에 저장합니다. 그런 다음 다시 스냅샷을 복원합니다.
etcdctl을 사용하여 서버에 연결하기 위해 다음 TLS 인증서/키가 제공됩니다
CA certificate: /etc/kubernetes/pki/etcd/ca.crt
Client certificate: /etc/kubernetes/pki/etcd/server.crt
Client key: /etc/kubernetes/pki/etcd/server.key

1. **https://kubernetes.io/ko/ 에서 ETCD backup 검색**

2. **root 계정으로 전환**

   sudo -i

```
cka@master:~$ sudo -i
[sudo] password for cka:
root@master:~#
```

3. **스냅샷 저장**

   ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 ₩ --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key ₩ snapshot save etcd-snapshot.db

```
root@master:~# ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save etcd-snapshot.db
{"level":"info","ts":1778500236.9146087,"caller":"snapshot/v3_snapshot.go:119","msg":"created temporary db file","path":"etcd-snapshot.db.part"
}
{"level":"info","ts":"2026-05-11T20:50:36.922332+0900","caller":"clientv3/maintenance.go:212","msg":"opened snapshot stream; downloading"}
{"level":"info","ts":1778500236.922404,"caller":"snapshot/v3_snapshot.go:127","msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info","ts":"2026-05-11T20:50:37.040561+0900","caller":"clientv3/maintenance.go:220","msg":"completed snapshot read; closing"}
{"level":"info","ts":1778500237.0605297,"caller":"snapshot/v3_snapshot.go:142","msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379","si
ze":"11 MB","took":0.145687708}
{"level":"info","ts":1778500237.0606096,"caller":"snapshot/v3_snapshot.go:152","msg":"saved","path":"etcd-snapshot.db"}
Snapshot saved at etcd-snapshot.db
```

4. **스냅샷 복원**

   ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 ₩ --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key ₩ snapshot restore etcd-snapshot.db

```
root@master:~# ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379   --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/e
tcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key   snapshot restore etcd-snapshot.db
{"level":"info","ts":1778500310.9045117,"caller":"snapshot/v3_snapshot.go:306","msg":"restoring snapshot","path":"etcd-snapshot.db","wal-dir":"
default.etcd/member/wal","data-dir":"default.etcd","snap-dir":"default.etcd/member/snap"}
{"level":"info","ts":1778500310.971445,"caller":"mvcc/kvstore.go:388","msg":"restored last compact revision","meta-bucket-name":"meta","meta-bu
cket-name-key":"finishedCompactRev","restored-compact-revision":20200}
{"level":"info","ts":1778500310.9808512,"caller":"membership/cluster.go:392","msg":"added member","cluster-id":"cdf818194e3a8c32","local-member
-id":"0","added-peer-id":"8e9e05c52164694d","added-peer-peer-urls":["http://localhost:2380"]}
{"level":"info","ts":1778500310.9903624,"caller":"snapshot/v3_snapshot.go:326","msg":"restored snapshot","path":"etcd-snapshot.db","wal-dir":"d
efault.etcd/member/wal","data-dir":"default.etcd","snap-dir":"default.etcd/member/snap"}
```

5. tree 설치

apt install tree

```
root@master:~# apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  tree
0 upgraded, 1 newly installed, 0 to remove and 34 not upgraded.
Need to get 47.4 kB of archives.
After this operation, 111 kB of additional disk space will be used.
Get:1 http://kr.archive.ubuntu.com/ubuntu noble-updates/universe amd64 tree amd64 2.1.1-2ubuntu3.24.04.2 [47.4 kB]
Fetched 47.4 kB in 0s (1,446 kB/s)
Selecting previously unselected package tree.
(Reading database ... 88522 files and directories currently installed.)
Preparing to unpack .../tree_2.1.1-2ubuntu3.24.04.2_amd64.deb ...
Unpacking tree (2.1.1-2ubuntu3.24.04.2) ...
Setting up tree (2.1.1-2ubuntu3.24.04.2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

6. 스냅샷 복원 결과 확인

tree default.etcd/member

```
root@master:~# tree default.etcd/member/
default.etcd/member/
├── snap
│   ├── 0000000000000001-0000000000000001.snap
│   └── db
└── wal
    └── 0000000000000000-0000000000000000.wal

3 directories, 3 files
```

[문제 1-2] Cluster Upgrade

마스터 노드의 모든 Kubernetes control plane및 node 구성 요소를 버전 1.34.7-1.1 버전으로 업그레이드 합니다.
master 노드를 업그레이드하기 전에 drain 하고 업그레이드 후에 uncordon해야 합니다.
"주의사항" 반드시 Master Node에서 root권한을 가지고 작업을 실행해야 한다

1. **https://kubernetes.io/ko/ 에서 kubedam upgrade 검색**

2. **쿠버네티스 노드의 버전 확인**

   Kubectl get no

```
cka@master:~$ kubectl get no
NAME       STATUS   ROLES           AGE   VERSION
master     Ready    control-plane   12d   v1.33.11
worker-1   Ready    <none>          12d   v1.33.11
worker-2   Ready    <none>          12d   v1.33.11
cka@master:~$
```

3. **Apt 업데이트 및 업그레이드 버전 결정**

   sudo apt update

   sudo apt-cache madison kubeadm

```
cka@master:~$ sudo apt-cache madison kubeadm
   kubeadm | 1.34.7-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.6-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.5-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.4-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.2-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
   kubeadm | 1.34.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb  Packages
```

4. **Kubeadm 업그레이드**

   sudo apt-mark unhold kubeadm && sudo apt-get update && sudo apt-get install -y kubeadm='1.34.7-1.1' && ₩ sudo apt-mark hold kubeadm

```
cka@master:~$ sudo apt-mark unhold kubeadm && \
> sudo apt-get update && sudo apt-get install -y kubeadm='1.34.7-1.1' && \
> sudo apt-mark hold kubeadm
Canceled hold on kubeadm.
Hit:1 http://kr.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://kr.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://kr.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Hit:5 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.34/deb  InRelease
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be upgraded:
  kubeadm
1 upgraded, 0 newly installed, 0 to remove and 47 not upgraded.
Need to get 12.6 MB of archives.
After this operation, 631 kB disk space will be freed.
Get:1 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.34/deb  kubeadm 1.34.7-1.1 [12.6 MB]
Fetched 12.6 MB in 1s (23.1 MB/s)
(Reading database ... 126541 files and directories currently installed.)
Preparing to unpack .../kubeadm_1.34.7-1.1_amd64.deb ...
Unpacking kubeadm (1.34.7-1.1) over (1.33.11-1.1) ...
Setting up kubeadm (1.34.7-1.1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.8.0-110-generic
Diagnostics:
  The currently running kernel version is not the expected kernel version 6.8.0-111-generic.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...

Service restarts being deferred:
 systemctl restart systemd-logind.service

No containers need to be restarted.

User sessions running outdated binaries:
 cka @ session #1: sshd[1743]
 cka @ session #4: sshd[5425]
 cka @ user manager service: systemd[1747]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kubeadm set on hold.
```

5. 업그레이드 버전 선택 후 실행

sudo kubeadm upgrade plan

sudo kubeadm upgrade apply v1.34.7-1.1

```
cka@master:~$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:
"", GitVersion:"v1.34.7", GitCommit:"66d3c15b9afd7f1aaa619156804202e52c897b84", GitTreeState:"clean", BuildDate:"2026-04-15T18:11:14Z", GoVe
rsion:"go1.25.9", Compiler:"gc", Platform:"linux/amd64"}
cka@master:~$ sudo kubeadm upgrade plan
[preflight] Running pre-flight checks.
[upgrade/config] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[upgrade/config] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
[upgrade] Running cluster health checks
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: 1.33.11
[upgrade/versions] kubeadm version: v1.34.7
I0512 18:35:38.717527   21209 version.go:260] remote version is much newer: v1.36.0; falling back to: stable-1.34
[upgrade/versions] Target version: v1.34.7
[upgrade/versions] Latest version in the v1.33 series: v1.33.11

Components that must be upgraded manually after you have upgraded the control plane with 'kubeadm upgrade apply':
COMPONENT   NODE       CURRENT    TARGET
kubelet     master     v1.33.11   v1.34.7
kubelet     worker-1   v1.33.11   v1.34.7
kubelet     worker-2   v1.33.11   v1.34.7

Upgrade to the latest stable version:

COMPONENT                 NODE      CURRENT    TARGET
kube-apiserver            master    v1.33.11   v1.34.7
kube-controller-manager   master    v1.33.11   v1.34.7
kube-scheduler            master    v1.33.11   v1.34.7
kube-proxy                          1.33.11    v1.34.7
CoreDNS                             v1.12.0    v1.12.1
etcd                      master    3.5.24-0   3.6.5-0

You can now apply the upgrade by executing the following command:

        kubeadm upgrade apply v1.34.7

_____________________________________________________________________


The table below shows the current state of component configs as understood by this version of kubeadm.
Configs that have a "yes" mark in the "MANUAL UPGRADE REQUIRED" column require manual config upgrade or
resetting to kubeadm defaults before a successful upgrade can be performed. The version to manually
upgrade to is denoted in the "PREFERRED VERSION" column.

API GROUP                 CURRENT VERSION   PREFERRED VERSION   MANUAL UPGRADE REQUIRED
kubeproxy.config.k8s.io   v1alpha1          v1alpha1            no
kubelet.config.k8s.io     v1beta1           v1beta1             no
_____________________________________________________________________
```

```
cka@master:~$ sudo kubeadm upgrade apply v1.34.7
[upgrade] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[upgrade] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
[upgrade/preflight] Running preflight checks
[upgrade] Running cluster health checks
[upgrade/preflight] You have chosen to upgrade the cluster version to "v1.34.7"
[upgrade/versions] Cluster version: v1.33.11
[upgrade/versions] kubeadm version: v1.34.7
[upgrade] Are you sure you want to proceed? [y/N]: y
[upgrade/preflight] Pulling images required for setting up a Kubernetes cluster
[upgrade/preflight] This might take a minute or two, depending on the speed of your internet connection
[upgrade/preflight] You can also perform this action beforehand using 'kubeadm config images pull'
[upgrade/control-plane] Upgrading your static Pod-hosted control plane to version "v1.34.7" (timeout: 5m0s)...
[upgrade/staticpods] Writing new Static Pod manifests to "/etc/kubernetes/tmp/kubeadm-upgraded-manifests3410658525"
[upgrade/staticpods] Preparing for "etcd" upgrade
[upgrade/staticpods] Renewing etcd-server certificate
[upgrade/staticpods] Renewing etcd-peer certificate
[upgrade/staticpods] Renewing etcd-healthcheck-client certificate
[upgrade/staticpods] Moving new manifest to "/etc/kubernetes/manifests/etcd.yaml" and backing up old manifest to "/etc/kubernetes/tmp/kubead
m-backup-manifests-2026-05-12-18-37-09/etcd.yaml"
[upgrade/staticpods] Waiting for the kubelet to restart the component
[upgrade/staticpods] This can take up to 5m0s
[apiclient] Found 1 Pods for label selector component=etcd
tmp/kubeadm-backup-manifests-2026-05-12-18-37-09/kube-scheduler.yaml"
[upgrade/staticpods] Waiting for the kubelet to restart the component
[upgrade/staticpods] This can take up to 5m0s
[apiclient] Found 1 Pods for label selector component=kube-scheduler
[upgrade/staticpods] Component "kube-scheduler" upgraded successfully!
[upgrade/control-plane] The control plane instance for this node was successfully upgraded!
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in the "kube-system" Namespace
[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with the configuration for the kubelets in the cluster
[upgrade/kubeconfig] The kubeconfig files for this node were successfully upgraded!
W0512 18:39:44.201391   21523 postupgrade.go:116] Using temporary directory /etc/kubernetes/tmp/kubeadm-kubelet-config923239509 for kubelet
config. To override it set the environment variable KUBEADM_UPGRADE_DRYRUN_DIR
[upgrade] Backing up kubelet config file to /etc/kubernetes/tmp/kubeadm-kubelet-config923239509/config.yaml
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[upgrade/kubelet-config] The kubelet configuration for this node was successfully upgraded!
[upgrade/bootstrap-token] Configuring bootstrap token and cluster-info RBAC rules
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get nodes
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post CSRs in order for nodes to get long term certificate credenti
als
[bootstrap-token] Configured RBAC rules to allow the csrapprover controller automatically approve CSRs from a Node Bootstrap Token
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all node client certificates in the cluster
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

[upgrade] SUCCESS! A control plane node of your cluster was upgraded to "v1.34.7".

[upgrade] Now please proceed with upgrading the rest of the nodes by following the right order.
```

6. 노드를 예약 불가능으로 표시하고 유지 관리 준비

kubectl drain master --ignore-daemonsets

```
cka@master:~$ kubectl drain master --ignore-daemonsets
node/master cordoned
Warning: ignoring DaemonSet-managed Pods: calico-system/calico-node-9p27x, calico-system/csi-node-driver-ngb4h, kube-system/kube-proxy-9vt6h
evicting pod tigera-operator/tigera-operator-747864d56d-ccdcq
evicting pod calico-system/calico-kube-controllers-6c5867d676-fp9wb
evicting pod calico-apiserver/calico-apiserver-7fd66d8b79-9zbbh
evicting pod calico-apiserver/calico-apiserver-7fd66d8b79-pzbls
evicting pod calico-system/goldmane-5f56496f4c-rjxhs
evicting pod calico-system/calico-typha-56f4d4f775-ls8nv
evicting pod calico-system/whisker-5ccd8cfbc-nw8qv
error when evicting pods/"calico-apiserver-7fd66d8b79-9zbbh" -n "calico-apiserver" (will retry after 5s): Cannot evict pod as it would viola
te the pod's disruption budget.
pod/tigera-operator-747864d56d-ccdcq evicted
pod/goldmane-5f56496f4c-rjxhs evicted
pod/calico-apiserver-7fd66d8b79-pzbls evicted
pod/calico-kube-controllers-6c5867d676-fp9wb evicted
pod/whisker-5ccd8cfbc-nw8qv evicted
pod/calico-typha-56f4d4f775-ls8nv evicted
evicting pod calico-apiserver/calico-apiserver-7fd66d8b79-9zbbh
error when evicting pods/"calico-apiserver-7fd66d8b79-9zbbh" -n "calico-apiserver" (will retry after 5s): Cannot evict pod as it would viola
te the pod's disruption budget.
evicting pod calico-apiserver/calico-apiserver-7fd66d8b79-9zbbh
error when evicting pods/"calico-apiserver-7fd66d8b79-9zbbh" -n "calico-apiserver" (will retry after 5s): Cannot evict pod as it would viola
te the pod's disruption budget.
evicting pod calico-apiserver/calico-apiserver-7fd66d8b79-9zbbh
pod/calico-apiserver-7fd66d8b79-9zbbh evicted
node/master drained
```

7. kubelet 및 kubectl 업그레이드

sudo apt-mark unhold kubelet kubectl && ₩ sudo apt-get update && sudo apt-get install -y kubelet='1.34.7-1.1' kubectl='1.34.7-1.1' && ₩ sudo apt-mark hold kubelet kubectl

```
cka@master:~$ sudo apt-mark unhold kubelet kubectl && \
> sudo apt-get update && sudo apt-get install -y kubelet='1.34.7-1.1' kubectl='1.34.7-1.1' && \
> sudo apt-mark hold kubelet kubectl
Canceled hold on kubelet.
Canceled hold on kubectl.
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.34/deb  InRelease
Hit:3 http://kr.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://kr.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://kr.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be upgraded:
  kubectl kubelet
2 upgraded, 0 newly installed, 0 to remove and 45 not upgraded.
Need to get 24.9 MB of archives.
After this operation, 22.3 MB disk space will be freed.
Get:1 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.34/deb  kubectl 1.34.7-1.1 [11.8 MB]
Get:2 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.34/deb  kubelet 1.34.7-1.1 [13.1 MB]
Fetched 24.9 MB in 1s (29.9 MB/s)
(Reading database ... 126541 files and directories currently installed.)
Preparing to unpack .../kubectl_1.34.7-1.1_amd64.deb ...
Unpacking kubectl (1.34.7-1.1) over (1.33.11-1.1) ...
Preparing to unpack .../kubelet_1.34.7-1.1_amd64.deb ...
Unpacking kubelet (1.34.7-1.1) over (1.33.11-1.1) ...
Setting up kubectl (1.34.7-1.1) ...
Setting up kubelet (1.34.7-1.1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.8.0-110-generic
Diagnostics:
  The currently running kernel version is not the expected kernel version 6.8.0-111-generic.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...
 systemctl restart kubelet.service

Service restarts being deferred:
 systemctl restart systemd-logind.service

No containers need to be restarted.

User sessions running outdated binaries:
 cka @ session #1: sshd[1743]
 cka @ session #4: sshd[5425]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kubelet set on hold.
kubectl set on hold.
```

8. kubelet 재시작

sudo systemctl daemon-reload

sudo systemctl restart kubelet

```
cka@master:~$ sudo systemctl daemon-reload
cka@master:~$ sudo systemctl restart kubelet
```

9. 노드 차단 해제

kubectl uncordon master

```
cka@master:~$ kubectl uncordon master
node/master uncordoned
```

10. 노드 버전 확인

kubectl get no

```
cka@master:~$ kubectl get no
NAME       STATUS   ROLES           AGE   VERSION
master     Ready    control-plane   13d   v1.34.7
worker-1   Ready    <none>          13d   v1.33.11
worker-2   Ready    <none>          13d   v1.33.11
```

[문제 2] 역할 기반 제어

[문제 2-1] Service Account, Role, RoleBinding 생성

애플리케이션 운영중 특정 namespace의 Pod들을 모니터할수 있는 서비스가 요청되었습니다. api-access 네임스페이스의 모든 pod를 view할 수 있도록 다음의 작업을 진행하시오.
1. api-access라는 새로운 namespace에 pod-viewer라는 이름의 Service Account를 만듭니다.
2. podreader-role이라는 이름의 Role과 podreader-rolebinding이라는 이름의 RoleBinding을 만듭니다.
3. 앞서 생성한 ServiceAccount를 API resource Pod에 대하여 watch, list, get을 허용하도록 매핑하시오

1. api-access라는 이름의 namespace 생성 및 확인

Kubectl create ns api-access

Kubectl get ns

```
cka@master:~$ kubectl create ns api-access
namespace/api-access created
cka@master:~$ kubectl get ns
NAME              STATUS   AGE
api-access        Active   6s
calico-apiserver  Active   11d
calico-system     Active   11d
default           Active   11d
kube-node-lease   Active   11d
kube-public       Active   11d
kube-system       Active   11d
tigera-operator   Active   11d
```

2. api-access라는 namespace에 pod-viewer라는 이름의 Service Account 생성

kubectl create sa pod-viewer -n api-access

```
cka@master:~$ kubectl create sa pod-viewer -n api-access
serviceaccount/pod-viewer created
```

3. Service Account 생성 확인

kubectl get sa -n api-access

```
cka@master:~$ kubectl get sa -n api-access
NAME         SECRETS   AGE
default      0         17m
pod-viewer   0         16m
```

4. **watch, list, get을 허용 하도록 podreader-role라는 이름의 Role 생성**

kubectl create role podreader-role -n api-access --resource=pod --verb=watch,list,get

```
cka@master:~$ kubectl create role podreader-role -n api-access --resource=pod --verb=watch,list,get
role.rbac.authorization.k8s.io/podreader-role created
cka@master:~$
```

```
cka@master:~$ kubectl get role -n api-access
NAME             CREATED AT
podreader-role   2026-05-11T08:35:55Z
```

5. **podreader-rolebinding이라는 이름의 RoleBinding 생성**

kubectl create rolebinding podreader-rolebinding --serviceaccount=api-access:pod-viewer --

role=podreader-role -n api-access

```
cka@master:~$ kubectl create rolebinding podreader-rolebinding --serviceaccount=api-acce
ss:pod-viewer --role=podreader-role -n api-access
rolebinding.rbac.authorization.k8s.io/podreader-rolebinding created
```

6. **RoleBinding 생성 확인**

Kubectl describe rolebinding -n api-access

```
cka@master:~$ kubectl get rolebinding -n api-access
NAME                    ROLE                   AGE
podreader-rolebinding   Role/podreader-role    14m
```

[문제 2-2] Service Account, ClusterRole, RoleBinding 생성

-애플리케이션 배포를 위해 새로운 ClusterRole을 생성하고 특정 namespace의 ServiceAccount를 바인드하시오.
-다음의 resource type에서만 Create가 허용된 ClusterRole deployment-clusterrole을 생성합니다.
-Resource Type: Deployment StatefulSet DaemonSet
-미리 생성된 namespace api-access에 cicd-token이라는 새로운 ServiceAccount를 만듭니다.
-ClusterRole deployment-clusterrole을 namespace api-access로 제한된 새 ServiceAccount cicd-token에 바인딩하세요

1. cicd-token이라는 새로운 ServiceAccount 생성

Kubectl create sa cicd-token -n api-access

```
cka@master:~$ kubectl create sa cicd-token -n api-access
serviceaccount/cicd-token created
```

2. ServiceAccount 생성 확인

Kubectl get sa -n api-access cicd-token

```
cka@master:~$ kubectl get sa -n api-access cicd-token
NAME         SECRETS   AGE
cicd-token   0         23h
```

3. Create가 허용된 ClusterRole deployment-clusterrole을 생성

kubectl create clusterrole deployment-clusterrole --resource=Deployment,StatefulSet,DaemonSet --verb=create

```
cka@master:~$ kubectl create clusterrole deployment-clusterrole --resource=deployment,st
atefulset,daemonset --verb=create
clusterrole.rbac.authorization.k8s.io/deployment-clusterrole created
```

4. ClusterRole 생성 확인

kubectl describe -n api-access clusterrole deployment-clusterrole

```
cka@master:~$ kubectl describe -n api-access clusterrole deployment-clusterrole
Name:         deployment-clusterrole
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources          Non-Resource URLs  Resource Names  Verbs
  ---------          -----------------  --------------  -----
  daemonsets.apps    []                 []              [create]
  deployments.apps   []                 []              [create]
  statefulsets.apps  []                 []              [create]
```

5. 생성한 clusterrole을 새 Service Account에 바인딩하도록 ClusterRoleBinding 생성

kubectl create clusterrolebinding deployment-clusterrole-rolebinding --serviceaccount=api-access:cicd-token --clusterrole=deployment-clusterrole -n api-access

```
cka@master:~$ kubectl create clusterrolebinding deployment-clusterrole-rolebinding --serviceaccount=api-access:ci
cd-token --clusterrole=deployment-clusterrole -n api-access
clusterrolebinding.rbac.authorization.k8s.io/deployment-clusterrole-rolebinding created
```

6. ClusterRoleBinding 생성 확인

kubectl describe clusterrolebinding -n api-access deployment-clusterrole-rolebinding

```
cka@master:~$ kubectl describe clusterrolebinding -n api-access deployment-clusterrole-rolebinding
Name:         deployment-clusterrole-rolebinding
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  deployment-clusterrole
Subjects:
  Kind            Name        Namespace
  ----            ----        ---------
  ServiceAccount  cicd-token  api-access
```

[문제 3] 노드 관리

[문제 3-1] 노드 비우기

Worker-2 노드를 스케줄링 불가능하게 설정하고, 해당 노드에서 실행 중인 모든 Pod을 다른 node로 reschedule 하세요.

1. 노드 확인

Kubectl get no

```
cka@master:~$ kubectl get no
NAME       STATUS   ROLES           AGE   VERSION
master     Ready    control-plane   13d   v1.33.11
worker-1   Ready    <none>          13d   v1.33.11
worker-2   Ready    <none>          13d   v1.33.11
```

2. Worker-2 에서 실행 중인 모든 pod를 다른 node로 reschedule 설정

kubectl drain worker-2 --ignore-daemonsets --force

```
cka@master:~$ kubectl drain worker-2 --ignore-daemonsets --force
node/worker-2 cordoned
error: unable to drain node "worker-2" due to error: cannot delete Pods with local storage (use --delete-emptydir-data to override): def
ault/weblog, continuing command...
There are pending nodes to be drained:
 worker-2
cannot delete Pods with local storage (use --delete-emptydir-data to override): default/weblog
```

3. 노드 확인

kubectl get no

```
cka@master:~$ kubectl get no
NAME       STATUS                     ROLES           AGE   VERSION
master     Ready                      control-plane   13d   v1.33.11
worker-1   Ready                      <none>          13d   v1.33.11
worker-2   Ready,SchedulingDisabled   <none>          13d   v1.33.11
```

4. Worker-2 drain 설정 해제

kubectl uncordon worker-2

```
cka@master:~$ kubectl uncordon worker-2
node/worker-2 uncordoned
```

5. 노드 확인

Kubectl get no

```
cka@master:~$ kubectl get no
NAME       STATUS   ROLES           AGE   VERSION
master     Ready    control-plane   13d   v1.33.11
worker-1   Ready    <none>          13d   v1.33.11
worker-2   Ready    <none>          13d   v1.33.11
```

[문제 3-2] Pod Scheduling

다음의 조건으로 pod를 생성하세요.
Name: eshop-store
Image: nginx
Nodeselector: disktype=ssd

1. https://kubernetes.io/ko/ 에서 nodeselector 검색

2. 노드 라벨 부여 및 확인 (worker-1 -> disktype=ssd, worker-2 -> disktype=hdd)

   Kubectl lable node worker-1 disktype=ssd

   Kubectl lable node worker-1 disktype=ssd

   kubectl get no -L disktype

```
cka@master:~$ kubectl label node worker-1 disktype=ssd
node/worker-1 labeled
cka@master:~$ kubectl label node worker-2 disktype=hhd
node/worker-2 labeled
```

```
cka@master:~$ kubectl get no -L disktype
NAME       STATUS   ROLES           AGE   VERSION    DISKTYPE
master     Ready    control-plane   13d   v1.33.11
worker-1   Ready    <none>          13d   v1.33.11   ssd
worker-2   Ready    <none>          13d   v1.33.11   hdd
```

3. 파드용 야물 파일 생성 (eshop-store)

   kubectl run eshop-store --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml > eshop-store.yaml

```
cka@master:~$ kubectl run eshop-store --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml > eshop-store.yaml
```

4. 야물 파일 수정

   vi eshop-strore.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
  name: eshop-store
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: eshop-store
  nodeSelector:
    disktype: ssd
~
```

5. 야물 파일 적용

   kubectl apply -f eshop-store.yaml

```
cka@master:~$ kubectl apply -f eshop-store.yaml
pod/eshop-store created
```

6. **파드 생성 확인**

kubectl get po eshop-store -o wide

```
cka@master:~$ kubectl get po eshop-store -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP              NODE       NOMINATED NODE   READINESS GATES
eshop-store   1/1     Running   0          99s   10.244.226.75   worker-1   <none>           <none>
```

[문제 4] 파드 생성

[문제 4-1] 환경변수, command, args 적용

'cka-exam'이라는 namespace를만들고, 'cka-exam' namespace에 아래와 같은 Pod를 생성하시오.
pod Name: pod-01
image: busybox
환경변수 : CERT = "CKA-cert"
command: /bin/sh
args: "-c", "while true; do echo $(CERT); sleep 10;done"

1. **https://kubernetes.io/ko/ 에서 command 검색**

2. **cka-exam이라는 namespace 생성**

kubectl create ns cka-exam

```
cka@master:~$ kubectl create ns cka-exam
namespace/cka-exam created
cka@master:~$ k get ns cka-exam
NAME       STATUS   AGE
cka-exam   Active   9s
```

3. **파드용 야물 생성**

kubectl run pod-01 -n cka-exam --image=ghcr.io/mjc-t/busybox --env=CERT="CKA-cert" --dry-run=client -o yaml > pod-01.yaml

```
cka@master:~$ kubectl run pod-01 -n cka-exam --image=ghcr.io/mjc-t/busybox --env=CERT="CKA-cert" --dry-run=client -o yaml > pod-01.yaml
```

4. **야물 파일 수정**

vi pod-01.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-01
  namespace: cka-exam
spec:
  containers:
  - env:
    - name: CERT
      value: CKA-cert
    image: ghcr.io/mjc-t/busybox
    name: pod-01
    command: ["/bin/sh"]
    args: ["-c", "while true; do echo $(CERT); sleep 10;done"]
```

5. **야물 파일 적용 및 파드 생성 확인**

Kubectl apply -f pod-01.yaml

Kubectl get po -n cka-exam

```
cka@master:~$ kubectl apply -f pod-01.yaml
pod/pod-01 created
cka@master:~$ kubectl get po -n cka-exam
NAME     READY   STATUS    RESTARTS   AGE
pod-01   1/1     Running   0          43s
```

[문제 4-2] Static pod 생성

> Worker-1 노드에 nginx-static-pod.yaml 라는 이름의 Static Pod를 생성하세요.
> pod name: nginx-static-pod
> image: nginx
> port : 80

1. **야물 파일 생성**

kubectl run nginx-static-pod --image=ghcr.io/mjc-t/nginx --port 80 --dry-run=client -o yaml > nginx-static-pod.yaml

```
cka@master:~$ kubectl run nginx-static-pod --image=ghcr.io/mjc-t/nginx --port 80 --dry-run=client -o yaml > nginx-static-pod.yaml
```

2. **야물 파일 수정**

vi nginx-static-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-pod
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx-static-pod
    ports:
    - containerPort: 80
~
```

3. **Worker-1의 static 위치 이동**

cd /etc/kubernetes/manifests

```
cka@worker-1:~$ cd /etc/kubernetes/manifests/
cka@worker-1:/etc/kubernetes/manifests$
```

4. **Worker-1에서의 static pod 생성을 위한 야물 파일 생성**

vi nginx-static-pod.yaml

```
cka@worker-1:/etc/kubernetes/manifests$ vi nginx-static-pod.yaml
```

5. **Master에 있는 nginx-static-pod.yaml 파일 내용 worker-1에 있는 파일로 복사 및 붙여넣기**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-pod
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx-static-pod
    ports:
    - containerPort: 80
~
```

6. **Master에서 야물 파일 적용**

kubectl apply -f nginx-static-pod.yaml

```
cka@master:~$ kubectl apply -f nginx-static-pod.yaml
pod/nginx-static-pod created
```

7. **Master에서 pod 생성 확인**

kubectl get po -o wide | grep nginx-static-pod-worker-1

```
cka@master:~$ kubectl get po -o wide | grep nginx-static-pod-worker-1
nginx-static-pod-worker-1       1/1     Running            0               21h   10.244.226.83    worker-1   <none>           <none>
```

[문제 4-3] 로그 확인

Pod " nginx-static-pod-worker-1"의 log를 모니터링하고, 메세지를 포함하는 로그라인을 추출하세요.
추출된 결과는 /home/cka/pod-log에 기록하세요.

1. 파드 확인

kubectl get po -o wide | grep nginx-static-pod-worker-1

```
cka@master:~$ kubectl get po -o wide | grep nginx-static-pod-worker-1
nginx-static-pod-worker-1       1/1     Running            0               21h   10.244.226.83    worker-1   <none>           <none>
```

2. nginx-static-pod-worker-1의 로그 모니터링

kubectl logs nginx-static-pod-worker-1

```
cka@master:~$ kubectl logs nginx-static-pod-worker-1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/11 11:18:29 [notice] 1#1: using the "epoll" event method
2026/05/11 11:18:29 [notice] 1#1: nginx/1.29.8
2026/05/11 11:18:29 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/11 11:18:29 [notice] 1#1: OS: Linux 6.8.0-110-generic
2026/05/11 11:18:29 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/11 11:18:29 [notice] 1#1: start worker processes
2026/05/11 11:18:29 [notice] 1#1: start worker process 29
2026/05/11 11:18:29 [notice] 1#1: start worker process 30
```

3. nginx-static-pod-worker-1의 로그 저장

kubectl logs nginx-static-pod-worker-1 > /home/cka/pod-log

```
cka@master:~$ kubectl logs nginx-static-pod-worker-1 > /home/cka/pod-log
```

4. 로그 확인

cat pod-log

```
cka@master:~$ cat pod-log
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/11 11:18:29 [notice] 1#1: using the "epoll" event method
2026/05/11 11:18:29 [notice] 1#1: nginx/1.29.8
2026/05/11 11:18:29 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/11 11:18:29 [notice] 1#1: OS: Linux 6.8.0-110-generic
2026/05/11 11:18:29 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/11 11:18:29 [notice] 1#1: start worker processes
2026/05/11 11:18:29 [notice] 1#1: start worker process 29
2026/05/11 11:18:29 [notice] 1#1: start worker process 30
```

[문제 4-4] Multi Container Pod 생성

> 4개의 컨테이너를 동작시키는 eshop-frontend Pod를 생성하시오.
> pod image: nginx, redis, memcached, busybox

1. **eshop-frontend Pod 용 야물 파일 생성**

   kubectl run eshop-frontend --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml > eshop-frontend.yaml

```
cka@master:~$ kubectl run eshop-frontend --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml > eshop-frontend.yaml
```

2. **야물 파일 수정**

   vi eshop-frontend.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: eshop-frontend
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx
  - image: ghcr.io/mjc-t/redis
    name: redis
  - image: ghcr.io/mjc-t/memcached
    name: memcached
  - image: ghcr.io/mjc-t/busybox
    name: busybox
~
```

3. **야물 파일 적용 및 확인**

   kubectl apply -f eshop-frontend.yaml

   kubectl get po -o wide | grep eshop-frontend

```
cka@master:~$ kubectl apply -f eshop-frontend.yaml
pod/eshop-frontend created
```

```
cka@master:~$ kubectl get po -o wide | grep eshop-frontend
```

[문제 5] 쿠버네티스 컨트롤러와 네트워킹

[문제 5-1] Rolling Update & Roll Back

Deployment를 이용해 nginx 파드를 3개 배포한 다음 컨테이너 이미지 버전을 rolling update하고 update record를 기록합니다. 마지막으로 컨테이너 이미지를 previous version으로 roll back 합니다.
name: eshop-payment
Image : nginx
Image version: 1.16
update image version: 1.17
label: app=payment, environment=production

1. 디플로이먼트용 야물 파일 생성

kubectl create deploy eshop-payment --image=ghcr.io/mjc-t/nginx:1.16 --replicas=3 --dry-run=client -o yaml > eshop-payment.yaml

```
cka@master:~$ kubectl create deploy eshop-payment --image=ghcr.io/mjc-t/nginx:1.16 --replicas=3 --dry-run=client -o yaml > eshop-payment.yaml
```

2. 야물 파일 수정

vi eshop-payment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: payment
    environment: production
  name: eshop-payment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payment
      environment: production
  template:
    metadata:
      labels:
        app: payment
        environment: production
    spec:
      containers:
      - image: ghcr.io/mjc-t/nginx:1.16
        name: nginx
~
```

3. 야물 적용 및 기록

kubectl apply -f eshop-payment.yaml –record

```
cka@master:~$ kubectl apply -f eshop-payment.yaml --record
Flag --record has been deprecated, --record will be removed in the future
deployment.apps/eshop-payment created
```

4. Image 버전 업데이트

kubectl set image deploy eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 –record

```
cka@master:~$ kubectl set image deploy eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record
Flag --record has been deprecated, --record will be removed in the future
deployment.apps/eshop-payment image updated
```

5. Image 버전 업데이트 기록 확인

kubectl rollout history deploy eshop-payment

```
cka@master:~$ kubectl rollout history deploy eshop-payment
deployment.apps/eshop-payment
REVISION  CHANGE-CAUSE
1         kubectl apply --filename=eshop-payment.yaml --record=true
2         kubectl set image deploy eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record=true
```

6. Image 버전 롤백

kubectl rollout undo deploy eshop-payment

```
cka@master:~$ kubectl rollout undo deploy eshop-payment
deployment.apps/eshop-payment rolled back
```

7. Image 버전 롤백 기록 확인

kubectl rollout history deploy eshop-payment

```
cka@master:~$ kubectl rollout history deploy eshop-payment
deployment.apps/eshop-payment
REVISION  CHANGE-CAUSE
2         kubectl set image deploy eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record=true
3         kubectl apply --filename=eshop-payment.yaml --record=true
```

[문제 5-2] Cluster IP

'devops' namespace에서 deployment eshop-order를 다음 조건으로 생성하시오.
- image: nginx, replicas: 2, label: name=order
'eshop-order' deployment의 Service를 만드세요.
Service Name: eshop-order-svc
Type: ClusterIP Port: 80

1. devops 라는 이름의 namespace 생성 및 확인

kubectl create ns devops

kubectl get ns devops

```
cka@master:~$ kubectl create ns devops
namespace/devops created
cka@master:~$ kubectl get ns devops
NAME      STATUS   AGE
devops    Active   111s
```

2. 디플로이먼트용 야물 파일 생성

kubectl create deploy eshop-order -n devops --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=client -o yaml > eshop-order.yaml

```
cka@master:~$ kubectl create deploy eshop-order -n devops --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=client -o yaml > eshop-order.yaml
```

3. 야물 파일 수정

vi eshop-order.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    name: order
  name: eshop-order
  namespace: devops
spec:
  replicas: 2
  selector:
    matchLabels:
      name: order
  template:
    metadata:
      labels:
        name: order
    spec:
      containers:
      - image: ghcr.io/mjc-t/nginx
        name: nginx
~
```

4. **야물 파일 적용**

kubectl apply -f eshop-order.yaml

```
cka@master:~$ kubectl apply -f eshop-order.yaml
deployment.apps/eshop-order created
```

5. **디플로이먼트 확인**

kubectl get deploy,po -n devops

```
cka@master:~$ kubectl get deploy,po -n devops
NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/eshop-order    2/2     2            2           54s

NAME                                 READY   STATUS    RESTARTS   AGE
pod/eshop-order-764ff544cd-kxhjh     1/1     Running   0          54s
pod/eshop-order-764ff544cd-xr689     1/1     Running   0          54s
```

6. **서비스 생성**

kubectl expose deploy eshop-order -n devops --name=eshop-order-svc --port=80 --target-port=80

```
cka@master:~$ kubectl expose deploy eshop-order -n devops --name=eshop-order-svc --port=80 --target-port=80
service/eshop-order-svc exposed
```

7. **서비스 생성 확인**

kubectl get svc -n devops

```
cka@master:~$ kubectl get svc -n devops
NAME              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
eshop-order-svc   ClusterIP   10.96.20.20    <none>        80/TCP    44s
```

[문제 5-3] NodePort

'front-end' deployment를 다음 조건으로 생성하시오.
image: nginx, replicas: 2, label: run=nginx
'front-end' deployment의 nginx 컨테이너를 expose하는'front-end-nodesvc'라는 새 service를 만듭니다.
Front-end로 동작중인 Pod에는 node의 30200 포트로 접속되어야 합니다.

1. **https://kubernetes.io/ko/ 에서 nodeport 검색**

2. **디플로이먼트용 야물 파일 생성**

   kubectl create deploy front-end --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=client -o yaml > front-end.yaml

```
cka@master:~$ kubectl create deploy front-end --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=clien
t -o yaml > front-end.yaml
```

3. **디플로이먼트용 야물 파일 수정**

   vi front-end.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    run: nginx
  name: front-end
spec:
  replicas: 2
  selector:
    matchLabels:
      run: nginx
  template:
    metadata:
      labels:
        run: nginx
    spec:
      containers:
      - image: ghcr.io/mjc-t/nginx
        name: nginx
```

4. **디플로이먼트용 야물 파일 적용**

   kubectl apply -f front-end.yaml

   kubectl get -f front-end.yaml

```
cka@master:~$ kubectl apply -f front-end.yaml
deployment.apps/front-end created
cka@master:~$ kubectl get -f front-end.yaml
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
front-end   2/2     2            2           14s
```

5. **서비스용 야물 파일 생성**

kubectl expose deploy front-end --name=front-end-nodesvc --port=80 --target-port=80 --

type=NodePort --dry-run=client -o yaml > front-end-nodesvc.yaml

```
cka@master:~$ kubectl expose deploy front-end --name=front-end-nodesvc --port=80 --target-port=80 --type=No
dePort --dry-run=client -o yaml > front-end-nodesvc.yaml
```

6. **서비스용 야물 수정**

vi front-end-nodesvc.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  labels:
    run: nginx
  name: front-end-nodesvc
spec:
  ports:
  - port: 80
    protocol: TCP
    targetPort: 80
    nodePort: 30200
  selector:
    run: nginx
  type: NodePort
```

7. **서비스용 야물 적용 및 확인**

kubectl apply -f front-end-nodesvc.yaml

kubectl get svc front-end-nodesvc

```
cka@master:~$ kubectl apply -f front-end-nodesvc.yaml
service/front-end-nodesvc created
cka@master:~$
cka@master:~$ kubectl get svc front-end-nodesvc
NAME                TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
front-end-nodesvc   NodePort   10.97.174.89   <none>        80:30200/TCP   26s
```

[문제 5-4] Pod Scale-out

실행되고 있는 eshop-order 디플로이먼트의 파드 숫자를 5개로 늘리시오.
- namespace: devops
- deployment: eshop-order
- replicas: 5

1. **네임스페이스 확인**

   kubectl get deploy -n devops

```
cka@master:~$ kubectl get deploy -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eshop-order   2/2     2            2           59s
```

2. **eshop-order 디플로이먼트의 파드 수 늘리기**

   kubectl scale deploy eshop-order -n devops --replicas=5

```
cka@master:~$ kubectl scale deploy eshop-order -n devops --replicas=5
deployment.apps/eshop-order scaled
```

3. **디플로이먼트 파드 수 확인**

   Kubectl get deploy eshop-order -n devops

```
cka@master:~$ kubectl get deploy eshop-order -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eshop-order   5/5     5            5           3m20s
```

[문제 6] NetworkPolicy & ingress

[문제 6-1] Network Policy

customera, customerb를 생성한 후, 각각 PARTITION=customera, PARTITION=customerb를 라벨링하시오.
default namespace에 다음과 같은 pod를 생성하세요.
name: poc
image: nginx
port: 80
label: app=poc
"partition=customera"를 사용하는 namespace에서만 poc의 80포트로 연결할 수 있도록 default namespace에 'allow-web-from-customera'라는 network Policy를 설정하세요. 보안 정책상 다른 namespace의 접근은 제한합니다.

1. **https://kubernetes.io/ko/ 에서 netpol 검색**

2. **네임스페이스 생성**

   kubectl create ns customera

   kubectl create ns customerb

```
cka@master:~$ kubectl create ns customera
namespace/customera created
cka@master:~$ kubectl create ns customerb
namespace/customerb created
```

3. **레이블 붙이기**

   kubectl label ns customera PARTITION=customera

   kubectl label ns customerb PARTITION=customerb

```
cka@master:~$ kubectl label ns customera PARTITION=customera
namespace/customera labeled
cka@master:~$ kubectl label ns customerb PARTITION=customerb
namespace/customerb labeled
```

4. **레이블 확인**

   kubectl get ns customera --show-labels

   kubectl get ns customerb --show-labels

```
cka@master:~$ kubectl get ns customera --show-labels
NAME        STATUS   AGE   LABELS
customera   Active   46s   PARTITION=customera,kubernetes.io/metadata.name=customera
cka@master:~$ kubectl get ns customerb --show-labels
NAME        STATUS   AGE   LABELS
customerb   Active   46s   PARTITION=customerb,kubernetes.io/metadata.name=customerb
```

5. **netpol.yaml 생성 및 수정 (1번에서 검색한 내용 중 사용할 yaml 파일 북사 후 붙여넣기)**

vi netpol.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-from-customera
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: poc
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          partition: customera
    ports:
    - protocol: TCP
      port: 80
```

6. **야물 파일 적용 및 확인**

kubectl apply -f netpol.yaml

kubectl get netpol

```
cka@master:~$ kubectl apply -f netpol.yaml
networkpolicy.networking.k8s.io/allow-web-from-customera created
cka@master:~$ kubectl get netpol
NAME                     POD-SELECTOR   AGE
allow-web-from-customera   app=poc        16s
```

[문제 6-2] Ingress

Create a new nginx Ingress resource as follows:
- Name: ping
- Namespace: ing-internal
- Exposing service hi on path /hi using service port 5678

1. https://kubernetes.io/ko/ 에서 ingress 검색

2. 네임스페이스 생성

   kubectl create ns ing-internal

```
cka@master:~$ kubectl create ns ing-internal
namespace/ing-internal created
```

3. Ingress용 야물 파일 생성 및 수정 (1번에서 검색한 내용 중 사용할 yaml 파일 북사 후 붙여넣기)

   vi ing.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ping
  namespace: ing-internal
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /hi
        pathType: Prefix
        backend:
          service:
            name: hi
            port:
              number: 5678
```

4. 야물 파일 적용 및 확인

   kubectl apply -f ing.yaml

   kubectl get ing -n ing-internal

```
cka@master:~$ kubectl apply -f ing.yaml
ingress.networking.k8s.io/ping created
cka@master:~$ kubectl get ing -n ing-internal
NAME   CLASS           HOSTS   ADDRESS   PORTS   AGE
ping   nginx-example   *                 80      13s
```

[문제 6-3] Service and DNS Lookup

image nginx를 사용하는 resolver pod를 생성하고 resolver-service라는 service를 구성합니다. 클러스터 내에서 service와 pod 이름을 조회할 수 있는지 테스트합니다.
- dns 조회에 사용하는 pod 이미지는 busybox:1.28이고, service와 pod 이름 조회는 nlsookup을 사용합니다.
- service 조회 결과는 /home/ubuntu/nginx.svc에 pod name 조회 결과는 /home/cka/nginx.pod 파일에 기록합니다.

1. 파드 생성 및 확인

kubectl run resolver --image=ghcr.io/mjc-t/nginx --port=80

kubectl get po resolver

```
cka@master:~$ kubectl run resolver --image=ghcr.io/mjc-t/nginx --port=80
pod/resolver created
cka@master:~$ kubectl get po resolver
NAME       READY   STATUS    RESTARTS   AGE
resolver   1/1     Running   0          8s
```

2. 서비스 생성 및 확인

kubectl expose pod resolver --name=resolver-service --port=80

kubectl get svc resolver-service

```
cka@master:~$ kubectl expose pod resolver --name=resolver-service --port=80
service/resolver-service exposed
cka@master:~$ kubectl get svc resolver-service
NAME               TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
resolver-service   ClusterIP   10.101.2.192   <none>        80/TCP    4s
```

3. 서비스 DNS 조회

kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10.101.2.192

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10.101.2.192
Server:         10.96.0.10
Address:        10.96.0.10:53

192.2.101.10.in-addr.arpa       name = resolver-service.default.svc.cluster.local

pod "test-nslookup" deleted
```

4. 파드 DNS 조회

kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10-99-62-43.default.pod.cluster.local

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10-101-2-192.default.pod.cluster.local
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   10-101-2-192.default.pod.cluster.local
Address: 10.101.2.192


pod "test-nslookup" deleted
```

5. 파일 저장

kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10.101.2.192 > nginx.svc

kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10-101-2-192.default.pod.cluster.local > nginx.pod

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10.101.2.192 > nginx.svc
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10-101-2-192.default.pod.cluster.local > nginx.pod
```

6. 파일 확인

cat nginx.svc

cat nginx.pod

```
cka@master:~$ cat nginx.svc
Server:         10.96.0.10
Address:        10.96.0.10:53

192.2.101.10.in-addr.arpa       name = resolver-service.default.svc.cluster.local

pod "test-nslookup" deleted
cka@master:~$ cat nginx.pod
Server:         10.96.0.10
Address:        10.96.0.10:53


Name:   10-101-2-192.default.pod.cluster.local
Address: 10.101.2.192

pod "test-nslookup" deleted
```

[문제 7] Resource 관리

[문제 7-1] emptyDir Volume

다음 조건에 맞춰서 nginx 웹서버 pod가 생성한 로그파일을 받아서 STDOUT으로 출력하는 busybox 컨테이너를 운영하시오.
Pod Name: weblog
Web container:
- Image: nginx:1.17
- Volume mount : /var/log/nginx
- Readwrite
Log container:
- Image: busybox
- args: /bin/sh, -c, "tail -n+1 -f /data/access.log"
- Volume mount : /data- readonly
emptyDir 볼륨을 통한 데이터 공유

1. **https://kubernetes.io/ko/ 에서 emptydir검색 후 Volumes으로 간 후 volumes 페이지에서 emptydir, readonly 검색**

2. **파드용 야물 파일 생성**

   kubectl run weblog --image=ghcr.io/mjc-t/nginx:1.17 --dry-run=client -o yaml > weblog.yaml

```
cka@master:~$ kubectl run weblog --image=nginx:1.17 --dry-run=client -o yaml > weblog.yaml
```

3. **야물 파일 수정**

   vi weblog.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: weblog
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx:1.17
    name: web
    volumeMounts:
    - mountPath: /var/log/nginx
      name: weblog
  - image: ghcr.io/mjc-t/busybox
    name: log
    args: [/bin/sh, -c, "tail -n+1 -f /data/access.log"]
    volumeMounts:
    - mountPath: /data
      name: weblog
      readOnly: true
  volumes:
  - name: weblog
    emptyDir: {}
```

4. **야물 파일 적용 및 확인**

kubectl apply -f weblog.yaml

kubectl get po -o wide | grep weblog

```
cka@master:~$ kubectl apply -f weblog.yaml
pod/weblog created
cka@master:~$ kubectl get po -o wide | grep weblog
weblog   2/2     Running   0          13s   10.244.226.104   worker-1   <none>           <none>
```

[문제 7-2] HostPath Volume

1. /data/cka/fluentd.yaml 파일을 만들어 새로은 Pod 생성하세요 신규생성 Pod Name: fluentd, image: fluentd, namespace: default)
2. 위 조건을 참고하여 다음 조건에 맞게 볼륨마운트를 설정하시오.
1. Worker node의 도커 컨테이너 디렉토리 : /var/lib/docker/containers 동일 디렉토리로 pod에 마운트하시오.
2. Worker node의 /var/log 디렉토리를 fluentd Pod에 동일이름의 디렉토리 마운트하시오.

1. **https://kubernetes.io/ko/ 에서 hostpath 검색 후 volumes로 이동 페이지 내 hostpath 검색**

2. **파드용 야물 파일 생성**

kubectl run fluentd --image=ghcr.io/mjc-t/fluentd --port=80 --dry-run=client -o yaml > fluentd.yaml

```
cka@master:~$ kubectl run fluentd --image=ghcr.io/mjc-t/fluentd --port=80 --dry-run=client -o yaml > fluentd.yaml
```

3. **야물 파일 수정**

vi fluentd.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: fluentd
spec:
  containers:
  - image: ghcr.io/mjc-t/fluentd
    name: fluentd
    ports:
    - containerPort: 80
    volumeMounts:
    - mountPath: /var/lib/docker/containers
      name: containerdir
    - mountPath: /var/log
      name: logdir
  volumes:
  - name: containerdir
    hostPath:
      path: /var/lib/docker/containers
  - name: logdir
    hostPath:
      path: /var/log
```

4. **야물 파일 적용 및 확인**

kubectl apply -f fluentd.yaml

kubectl get po -o wide | grep fluentd

```
cka@master:~$ kubectl apply -f fluentd.yaml
pod/fluentd created
cka@master:~$ kubectl get po -o wide | grep fluentd
fluentd   1/1     Running   0          17s     10.244.133.214   worker-2   <none>           <none>
```

[문제 7-3] Persistenet Volume

> pv001라는 이름으로 size 1Gi, access mode ReadWriteMany를 사용하여 persistent volume을 생성합니다.
> volume type은 hostPath이고 위치는 /tmp/app-config입니다.

1. **https://kubernetes.io/ko/ 에서 pv 검색 후 이동, 페이지 내에서 pv00, hostpath 검색**

2. **pv용 야물 파일 생성 및 수정 (1번에서 검색한 내용 중 사용할 yaml 파일 북사 후 붙여넣기)**

vi pv001.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  hostPath:
      path: /tmp/app-config
```

3. **야물 파일 적용**

kubectl apply -f pv001.yaml

```
cka@master:~$ kubectl apply -f pv001.yaml
persistentvolume/pv0001 created
```

4. **pv 확인**

kubectl get pv pv001.yaml

```
cka@master:~$ kubectl get pv pv0001
NAME     CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM              STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
pv0001   1Gi        RWO            Retain           Bound    default/pv-volume                 <unset>                          96s
cka@master:~$
```

[문제 7-4] PVC를 사용하는 애플리케이션 Pod 운영

다음의 조건에 맞는 새로운 PersistentVolumeClaim 생성하시오.
Name: pv-volume
Capacity: 10Mi
앞서 생성한 pv-volume PersistentVolumeClaim을 mount하는 Pod를 생성하시오.
Name: web-server-pod
Image: nginx
Mount path: /usr/share/nginx/html Volume에서 ReadWriteMany 액세스 권한을 가지도록 구성합니다.

1. **https://kubernetes.io/ko/ 에서 pv 검색 후 이동, 페이지 내에서 kind: persistent, storage class 검색**

2. **Pvc용 야물 파일 생성 및 수정 (1번에서 검색한 내용 중 사용할 yaml 파일 북사 후 붙여넣기)**

   vi pvc.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pv-volume
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Mi
```

3. **Pvc용 야물 파일 적용**

   kubectl apply -f pvc.yaml

```
cka@master:~$ kubectl apply -f pvc.yaml
persistentvolumeclaim/pv-volume created
```

4. **https://kubernetes.io/ko/ 에서 pv 검색 후 이동, 페이지 내에서 kind: pod 검색**

5. **Pod용 야물 생성 및 수정 (1번에서 검색한 내용 중 사용할 yaml 파일 북사 후 붙여넣기)**

   vi web-server-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-server-pod
spec:
  containers:
    - name: myfrontend
      image: ghcr.io/mjc-t/nginx
      volumeMounts:
      - mountPath: "/usr/share/nginx/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: pv-volume
```

6. **pod용 야물 적용 및 확인**

kubectl apply -f web-server-pod.yaml

kubectl get po -o wide | grep web-server-pod

```
cka@master:~$ kubectl apply -f web-server-pod.yaml
pod/web-server-pod created
```